

Pure Rho and Cost: One Authored Calculus, Named Extensions, and Corrected Hypotheses

Mettapedia

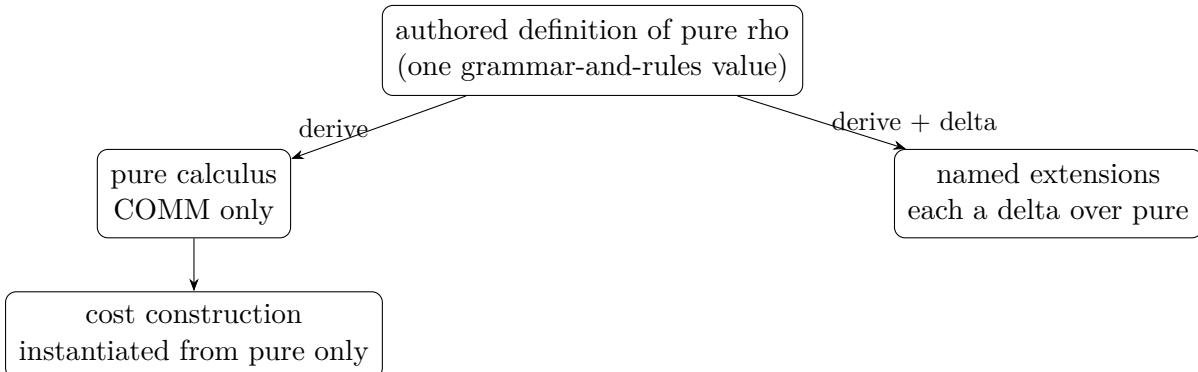
July 2026 (working paper)

Abstract

The rho-calculus is a process calculus in which every name is a quoted process. This paper fixes the exact calculus we formalize and implement: a *pure core* whose only operational reduction is communication, with every engine convenience recorded as a named extension derived from that core, never mixed into it. On this base we state the cost construction — wrapping terms and gating reductions by funding — together with two corrections to its published hypotheses: the injectivity assumption behind *quote-faithfulness* cannot hold for ordinary monad multiplication on any nontrivial signature monoid: joint injectivity of multiplication together with a two-sided unit forces the carrier to be a subsingleton. The repair is not a stronger monoid hypothesis. Funded execution instead forms an Atkey-style parameterized monad indexed by complete pre- and post-resource states. Exact causal receipts are retained for authority decisions, while prices and budget totals factor through explicit, generally non-injective aggregation maps. The obstruction, the parameterized monad laws, this authority/observation factorization, and a computable canonical section for pure structural congruence are formalized. On the binder-safe funded fragment, initialization and reduction preserve scope, and every declarative funded path erases to a path of the established pure-rho GSLT with exactly one COMM step per funded event. Claims are marked as formalized or open; nothing here is asserted beyond its marker.

1 Introduction

A calculus and its implementation drift apart unless the boundary between them is a maintained artifact. Reference implementations of the rho-calculus add conveniences — direct execution of quoted processes, polyadic inputs, name extrusion — that are individually reasonable and collectively fatal to the question “what exactly has been proved?” Our answer is architectural:



One authored definition is the sole semantic root. The pure calculus and each extension are derived from it; extensions import the pure theory and add a delta, and the pure theory never imports an extension. The cost profile of the runtime is instantiated from the pure calculus alone. The direction of these imports is enforced structurally (by module dependencies and by theorems), not merely by convention.

2 The pure calculus

Syntax. Processes P and names x are mutually defined:

$$P ::= 0 \mid \text{for}(y \leftarrow x) P \mid x!(P) \mid P \mid P \mid *x \qquad x ::= @P.$$

A name is a quoted process $@P$; the process $*x$ is the *drop* of a name. Quotation and dropping are the calculus's reflection: processes can be passed as data and data can be run — but, as we now make precise, only through communication.

Two equivalences, kept separate. The calculus has two distinct static equivalences, and sorting laws into the correct one is load-bearing:

- *Name equivalence* contains the cancellation $@(*x) \equiv_N x$: quoting the drop of a name gives back the name. This is an equation between *names*.
- *Process structural congruence* contains alpha-equivalence and the commutative-monoid laws for parallel composition ($P|Q \equiv Q|P$, $(P|Q)|R \equiv P|(Q|R)$, $P|0 \equiv P$).

Neither equivalence contains a rule that *runs* anything. In particular, $*(@P) \rightarrow P$ is *not* a law of the pure calculus, neither as an equation nor as a reduction.

One reduction. The only operational rule is communication:

$$\text{for}(y \leftarrow x) P \mid x!(Q) \longrightarrow P\{@Q/y\},$$

where the substitution is semantic: when $P\{@Q/y\}$ replaces y inside a drop, $*y$ becomes Q . Dequotation therefore happens *only* through substitution after a communication — you can run exactly the code you were sent.

A positive and a negative example.

$$\underbrace{\text{for}(y \leftarrow @0) (*y) \mid @0!(x!(0))}_{\text{reducible}} \longrightarrow x!(0) \qquad \underbrace{*(@x!(0))}_{\text{stuck in pure rho}} \not\longrightarrow$$

The left process communicates on the name $@0$ and the received quoted process runs via substitution. The right process is a free-standing drop of a quotation; in the pure calculus it is inert. **[formalized]** for the concrete one-step relation (communication is its only constructor); the corresponding negative example is part of the extension test suite described in Section 4.

3 Running quoted code without extending the calculus

The convenience that tempts implementations to add a primitive $*(@P) \rightarrow P$ step is already expressible, at the cost of one honest communication, by a two-process *runner* idiom:

$$run!(@P) \mid \text{for}(y \leftarrow run)(*y) \longrightarrow P.$$

The drop fires through substitution on a received name — pure rho — and the execution of a quoted process costs exactly one communication event, which the cost semantics of Section 5 meters honestly. Nothing about expressivity forces an execution primitive into the calculus; adding one changes the step relation, the stuck states, and every cost ledger, in exchange for saving a single COMM.

4 Extensions as named deltas

Engine conveniences are legitimate — as *named* extensions. The discipline has three parts.

Structure. Each extension is a delta over the pure theory: the extension module imports the pure calculus and adds constructors, equations, or rules; the pure modules import no extension. Distinct conveniences are distinct deltas — at minimum, a minimal *execution* extension (pure rho plus $*(@P) \rightarrow P$) is kept separate from a *reference-implementation profile* (polyadic input, replicated input, name extrusion, native data operations), because they change different parts of the theory and generate different cost events. In the formal library these deltas may be collected in one clearly labelled extended-rho namespace; the runtime’s cost profile instantiates none of them.

Theorems, not conventions. Each delta carries a positive and a negative theorem: the free drop *cannot* step in pure rho [**formalized**]; the free drop *does* step in the execution extension [**formalized**]; and the cost profile is instantiated from the pure calculus specifically. The two reduction claims are proved for the declarative language definitions. The runtime import/profile boundary remains a separate closure gate. A text search for stray constructors is useful supplementary evidence; the load-bearing boundary is the import graph and these theorems.

The delta as a document. A reference implementation’s distance from the paper calculus is itself worth recording. The audit of the implementation we compare against (the *mettail* engine’s rho language) yields exactly such a delta: an execution rewrite, polyadic and persistent communication, extrusion, and native data — all useful, none of them part of the calculus whose metatheory we prove. Recording the delta turns “the paper and the code disagree” from folklore into data.

5 The cost construction: corrected hypotheses

The cost construction wraps terms and gates reduction by funding: a wrapped redex fires only when the wrapper supplies a token drawn from a commutative monoid $(S, *, e)$ of *signatures*, and each reduction consumes its funding and emits a receipt. Runtime aspects — receipts, budgets, parallel execution — are treated in the companion engine paper; here we state the mathematical hypotheses, two of which require correction.

5.1 The quote-faithfulness obstruction

Call a map *quote-faithful* when it is injective on the signature keys reachable in a given theory. The published development uses per-argument injectivity of the signature operation. Three separate claims hide under this phrase, with three different fates.

(a) Fixed-element level. “Multiplication by a fixed s is injective” ($s * a = s * b \Rightarrow a = b$) is exactly *cancellativity* of the monoid — and it *fails* in general:

$$\max(3, 1) = \max(3, 2) \qquad \infty + 1 = \infty + 2.$$

Idempotent (tropical) aggregation and saturating budgets are therefore outside the hypothesis. Free signatures — finite multisets of generators — are cancellative, so the intended examples survive *at this fixed-element level once the hypothesis is stated*. **[formalized]**

(b) Multiplication level. The monad multiplication $\mu : \text{Cost}^2 \rightarrow \text{Cost}$ flattens two layers of wrapping, combining an outer and an inner signature into one key. Cancellativity does *not* make this flattening injective, because joint injectivity of $(a, b) \mapsto a * b$ is a unique-factorization property, not a cancellation property:

$$1 + 2 = 0 + 3 \quad \text{in } (\mathbb{N}, +), \qquad \{a\} \uplus \{b, c\} = \{a, b\} \uplus \{c\} \quad \text{for free multisets.}$$

Both monoids are cancellative; in both, distinct outer/inner factorizations flatten to the same key. More generally, if a binary operation with a two-sided unit were jointly injective, then $(e, a) = (a, e)$ for every a , forcing every element to equal e . Thus:

$$\text{inj}((a, b) \mapsto a * b) \iff S \text{ is a subsingleton.}$$

No nontrivial monoid—including words, free multisets, or natural-number costs—can satisfy the published multiplication-level requirement. This is an impossibility theorem, not a missing hypothesis. **[formalized]**

(c) Unit level. The unit η must not fabricate funding. In the corrected semantics, unit is the empty execution at a fixed resource state and carries the empty receipt. Every nonempty funded execution advances the event counter; hence no nonempty execution can masquerade as an identity. The ordinary “freely available unit token” argument is withdrawn. **[formalized]** for the concrete funded semantics.

5.2 The typed repair: pre/post indexed execution

Let I be the type of complete funded states: event counter, live resource components, provenance, and the proofs required to continue safely. For $i, j \in I$, let $T(i, j, X)$ be the type of funded executions from i to j returning a value in X . The operations are

$$\text{pure}_i : X \rightarrow T(i, i, X), \qquad \text{bind} : T(i, j, X) \times (X \rightarrow T(j, k, Y)) \rightarrow T(i, k, Y).$$

The shared middle index is part of the type. An execution ending at j cannot be composed with one beginning at a different resource state; invalid funding composition is therefore untypeable. Empty execution supplies `pure`, path concatenation supplies `bind`, and the two unit laws and associativity are proved from the corresponding path laws. **[formalized]**

This is an Atkey-style parameterized monad [5]. It is also the pre/post-state specialization of the broader graded and parametric-effect perspective [6, 7]: a monoid grade summarizes effects, while a category grade records which state transitions may compose. The present formalization constructs the resource transition category and the parameterized monad instance; it does not claim a fully general category-graded monad for arbitrary result families.

5.3 Authority is exact; pricing is an observation

A causal receipt retains event occurrences, funding records, and their order. Sequential composition concatenates these exact receipts. Budget totals, additive prices, multiplicative prices, and signature multisets are obtained only afterward by monoid homomorphisms:

$$\text{Receipt} \longrightarrow \text{Aggregate} \longrightarrow \text{Price}.$$

The first object is suitable for replay and authority decisions; the later objects intentionally forget information. In particular, the raw account functor is proved to factor through the exact authority-receipt functor and an explicit aggregation homomorphism. The aggregation is not injective, and no total reconstruction from it exists. **[formalized]**

5.4 What stands

The concrete runtime already follows the corrected shape: it retains receipts, gates every firing by actual funding, and derives totals by folding. Its receipt replay, sequential/threaded agreement, and sanitizer evidence do not depend on joint injectivity of signature multiplication. What is withdrawn is the ordinary-monad faithfulness claim. What replaces it is stronger where authority matters: exact indexed transitions and exact receipts, with every lossy view named as an observation.

6 Canonical forms and the computable section

Signature keys, and every quotient used above, presuppose a *computable canonical form* for the two static equivalences of the pure calculus. A canonicalizer `nf` must:

- normalize binders so that alpha-equivalent terms coincide (locally nameless or de Bruijn representation);
- normalize processes and names mutually, with termination measured by quote depth;
- orient the name cancellation $@(*x) \equiv_N x$ as a directed simplification;
- flatten parallel composition, remove 0, normalize components recursively, and sort them under a proved total order on terms;
- keep name equivalence and process congruence as *separate* sorted relations — the canonical form respects both without merging them into one relation over raw syntax.

The obligations are soundness ($\text{nf}(P) \equiv P$), completeness on the pure carrier ($P \equiv Q \Rightarrow \text{nf}(P) = \text{nf}(Q)$), and idempotence. **[formalized]** The proved carrier consists of patterns without the optional finite-set extension. Canonicalization preserves that subtype and descends by quotient lifting to a computable representative function; the representative is proved to belong to its original equivalence class. The continued-cut presentation now stores this explicit representative carrier and embedding instead of selecting representatives with `Quotient.out`.

The mathematical key is the canonical form itself. A runtime digest may later refine it under an explicitly assumed collision-resistance property, and the formalization never treats a concrete hash function as injective.

7 One authored presentation and semantic agreement

The pure syntax, equations, reflective constructors, and COMM rule are authored once in the rho language definition. The derived well-sorted judgments supply the induction principles for names, processes, and parallel lists. On that derived process carrier, reflective substitution compiled from the authored declaration is proved exactly equal to the independently developed semantic COMM substitution. Therefore every well-sorted COMM contractum compiled from the authored rule is a step of the established rho GSLT. **[formalized]**

The restriction to the derived process judgment is essential. The shared raw pattern type also contains ill-sorted terms, including quotations placed directly in process position; no global agreement theorem is claimed for those terms. During the proof, this restriction exposed and led to the repair of an overlapping unary-pattern dispatcher that could descend beneath a quotation. A regression example now fixes that boundary.

8 Scoped funded execution refines pure rho

The raw de Bruijn representation is intentionally larger than the object language: it can express an index that is outside its enclosing binder, and it can retain such an index underneath a quotation. These values are useful as negative tests, but they are not rho terms. Following the standard scope-safe-syntax discipline [8], the funded syntax has a derived *binder-safe* judgment. A quotation seals the surrounding binder, so a term quoted inside a receiver body must already be closed with respect to that body. The judgment is derived over the existing cost syntax; it neither adds a second syntax nor changes COMM.

The refinement result has the usual initialization–preservation–simulation shape. First, flattening a binder-safe source term into a funded configuration preserves binder safety. Second, every declarative funded step maps a binder-safe configuration to a binder-safe configuration. Third, cost substitution on this fragment agrees, up to the *pure StructuralCongruence*, with the established semantic COMM substitution. The proof uses the pure canonical section and never an equivalence containing the executable free-Drop rule. **[formalized]**

Configuration erasure is defined by quotient lifting over permutation of funded components, never by selecting a representative with `Quotient.out`. Each binder-safe `CostStep` then erases to one `rhoGSLT.Step`. Induction over declarative paths gives a `rhoGSLT.RewritePath`; its length is exactly the number of funded events and exactly the length of the path’s ordered demand receipt. Thus erasure inserts neither administrative reductions nor executable-Drop reductions. **[formalized]**

This is a theorem about the declarative funded relation. The compiled C engine is checked independently by differential execution and Lean receipt replay, including rejection of tampered receipts. That evidence is deliberately not presented as a universal theorem about compiled C or about every value accepted by the low-level raw parser.

9 Provenance and status

The formalization pins the exact manuscript versions it implements — title, date, and content digest recorded alongside the theory — because a silently updated source would change the meaning of proved names. The operative sources are the reflective higher-order calculus paper of Meredith and Radestock; the graph-enriched Lawvere-theory account of operational semantics of Stay and Meredith; and the June 2026 versions of the continued-interactive-GSLT/cost and cost-accounted-rho manuscripts, whose earlier circulated revisions differ materially and are kept only as comparison history.

Item	Section	Status
COMM-only concrete one-step relation	§2	formalized
Free drop inert in pure rho (negative theorem)	§2, §4	formalized
Runner idiom (one-COMM execution)	§3	stated; example-level
Execution delta with positive/negative theorems	§4	formalized
Fixed-element cancellation boundary	§5(a)	formalized
μ joint-injectivity impossibility	§5(b)	formalized
Parameterized monad laws on funded execution	§5	formalized
Exact receipt / lossy aggregate factorization	§5	formalized
Computable pure canonical form + section	§6	formalized
Authored COMM compiler agreement with rho GSLT	§7	formalized
Binder-safe initialization and preservation	§8	formalized
Funded-step erasure to pure rho GSLT	§8	formalized
Declarative funded paths to pure rewrite paths, exact length	§8	formalized
Pure/extended and cost-profile execution boundary	§4	Lean theorems; C differential gate

References

- [1] L. G. Meredith and M. Radestock. A reflective higher-order calculus. *Electronic Notes in Theoretical Computer Science*, 141(3):49–67, 2005.
- [2] M. Stay and L. G. Meredith. Representing operational semantics with enriched Lawvere theories. 2017.
- [3] L. G. Meredith. Continued interactive GSLTs and the cost endofunctor: wrapping by construction; and Cost accounting for the reflective higher-order calculus. Manuscripts, June 2026 versions (pinned by digest in the repository).
- [4] F1R3FLY-io. *mettail*: a reference implementation family for rho-style calculi. Open-source repository; used here as a bounded encoding reference, with its implementation delta recorded in Section 4.
- [5] R. Atkey. Parameterised notions of computation. *Journal of Functional Programming*, 19(3–4):335–376, 2009. <https://bentnib.org/param-notions.html>
- [6] S. Katsumata. Parametric effect monads and semantics of effect systems. In *Proceedings of POPL*, 2014. <https://group-mmm.org/~s-katsumata/paper/pp12014.pdf>

- [7] D. Orchard, P. Wadler, and H. Eades III. Unifying graded and parameterised monads. In *Proceedings of MSFP*, 2020. <https://arxiv.org/abs/2001.10274>
- [8] G. Allais, R. Atkey, J. Chapman, C. McBride, and J. McKinna. A type- and scope-safe universe of syntaxes with binding: their semantics and proofs. 2021. <https://arxiv.org/abs/2001.11001>